COLLIFY

A Smart Classroom Management System

Ria Khurana

Arun

Abstract

Collify is a modern, full-stack web-based Classroom and Testing Platform designed to bridge the gap between traditional classroom management and effective student assessment. The system provides a unified digital environment where teachers can create and manage classrooms, share announcements and learning materials, conduct MCQ-based quizzes with automatic grading, and track student attendance.

Students can join classrooms using unique join codes, access learning resources, attempt quizzes in real-time, and view their performance and attendance records. The platform implements a clear role-based access control mechanism, ensuring teachers have full control over content creation and assessment, while students have a focused learning and evaluation experience.

Built using Next.js for the frontend, Node.js and Express for the backend, and MongoDB as the database, Collify emphasizes simplicity, scalability, and real-time feedback. Key features include JWT-based authentication, secure file uploads for study materials, and a flexible post system supporting different content types such as announcements, materials, and Google Meet links.

The primary objective of Collify is to create an intuitive, efficient, and engaging learning ecosystem that reduces the dependency on multiple tools and enhances the overall teaching-learning process.

Table of Contents

1. Introduction

i) Purpose

The primary objective of **Collify** is to bridge the gap between traditional classroom management and modern digital efficiency. In an era where hybrid learning is the norm, Collify provides a centralized hub where teachers can organize resources and students can access them without the clutter of generic social media or fragmented email chains. The platform specifically targets the automation of repetitive tasks—such as grading multiple-choice questions (MCQs) and tracking attendance—allowing educators to focus more on instruction and less on administration.

ii) Scope

Collify is a comprehensive Learning Management System (LMS) tailored for academic environments. The scope includes:

- **Secure Authentication:** Role-based access for Teachers and Students using industry-standard protocols.
- **Virtual Environments:** Creation and management of digital classrooms via unique join codes.
- **Content Distribution:** Seamless sharing of announcements, study materials, and Google Meet links.
- **Assessment Engine:** A robust MCQ testing platform featuring auto-grading and real-time student feedback.
- **Planned Scalability:** Future modules include a coding judge for DSA problems and a digital attendance tracker to provide a holistic academic record.

iii) Overview

The platform is built on a decoupled architecture, utilizing a **Next.js** frontend for a fast, responsive user experience and a **Node.js/Express** backend for scalable API management. By leveraging a "Neon-Blue" arcade aesthetic, Collify aims to reduce the "fatigue" often associated with dull educational software, making the learning process more engaging for students while remaining professional and powerful for teachers.

---

2. System Requirements

i) Functional Requirements

These define the core actions the system must support:

- **Role Identification:** The system must distinguish between 'Teacher' and 'Student' roles during registration to gatekeep administrative features.
- **Classroom Lifecycle:** Teachers must be able to create, update, and delete classrooms. Students must be able to join via a 6-digit alphanumeric code.
- **Communication Hub:** The system must allow teachers to create posts (announcements/materials) that are instantly visible to all joined students.
- **Quiz Engine:** Teachers require a form-based interface to generate quizzes. The system must automatically compare student submissions against a "correct answer" key stored in the database.
- **Secure Session Management:** Users must stay logged in via JWT, ensuring they only see data relevant to their specific classrooms.

ii) Non-Functional Requirements

These define the quality attributes of the system:

- **Security:** Sensitive data, specifically passwords, must be hashed using bcrypt before storage. APIs must be protected from unauthorized access via Middleware. JWT for secure API interactions.
- **Performance:** Next.js Server Components should be used where possible to ensure sub-2-second page loads.
- **Responsiveness:** The UI must be fully functional on both desktop (for quiz creation) and mobile (for quiz attempts).
- **Reliability:** The MongoDB connection must include retry logic to ensure data persistence even during minor network fluctuations.
- **Maintainability:** Mongoose for schema management ensures easy updates and feature additions.

iii) Technical Stack

Backend:

- Node.js : Javascript runtime for building the server.
- Express: Web framework for Node.js to manage routing and server-side logic.

- JWT (JSON Web Token): Used for implementing authentication and authorization.
- bcrypt: For securely hashing passwords.
- Mongoose: Object-Document Mapper (ODM) for MongoDB to interact with the database.
- MongoDB Atlas: Cloud-based database service for storing your application's data.

Frontend:

- Next.js 14 (App Router) + React. Chosen for its optimized routing and SEO capabilities.

Cloud/Hosting:

- AWS: Cloud services for hosting, storage and other infrastructure needs.
- Cloudinary : For Storing pdf files on cloud

---

3. System Design

3.1 ER Diagram

The Entity-Relationship (ER) Diagram represents the logical structure of the Collify database. It defines the main entities, their attributes, and the relationships between them.

Entities and Key Attributes:

- **User** Attributes: user_id (Primary Key), username, email, password, role (ENUM: 'student', 'teacher')
- **Classroom** Attributes: classroom_id (Primary Key), name, joinCode, teacher_id (Foreign Key referencing User)
- **Post** Attributes: post_id (Primary Key), classroom_id (Foreign Key), author_id (Foreign Key), type (ENUM: announcement, material, google_meet, quiz), title, content, attachments (Array of URLs), createdAt
- **Attendance** Attributes: attendance_id (Primary Key), classroom_id (Foreign Key), date, records (JSON array storing student-wise attendance status)
- **Quiz** Attributes: quiz_id (Primary Key), classroom_id (Foreign Key), title, totalMarks
- **Attempt** Attributes: attempt_id (Primary Key), student_id (Foreign Key), quiz_id (Foreign Key), score, answers (JSON)

Relationships:

- One **User** (Teacher) can create **many Classrooms** (1:N)
- One **Classroom** can have **many Students** enrolled (N:N relationship implemented via array in Classroom)
- One **Classroom** can have **many Posts** (1:N)
- One **Classroom** can have **many Quizzes** (1:N)
- One **Classroom** can have **many Attendance records** (1:N)
- One **Quiz** can have **many Attempts** by students (1:N)
- One **User** (Student) can make **many Attempts** (1:N)

The ER Diagram illustrates a robust relational structure that supports role-based access, efficient classroom management, content sharing, and assessment features. It ensures data integrity through proper foreign key constraints and normalization.

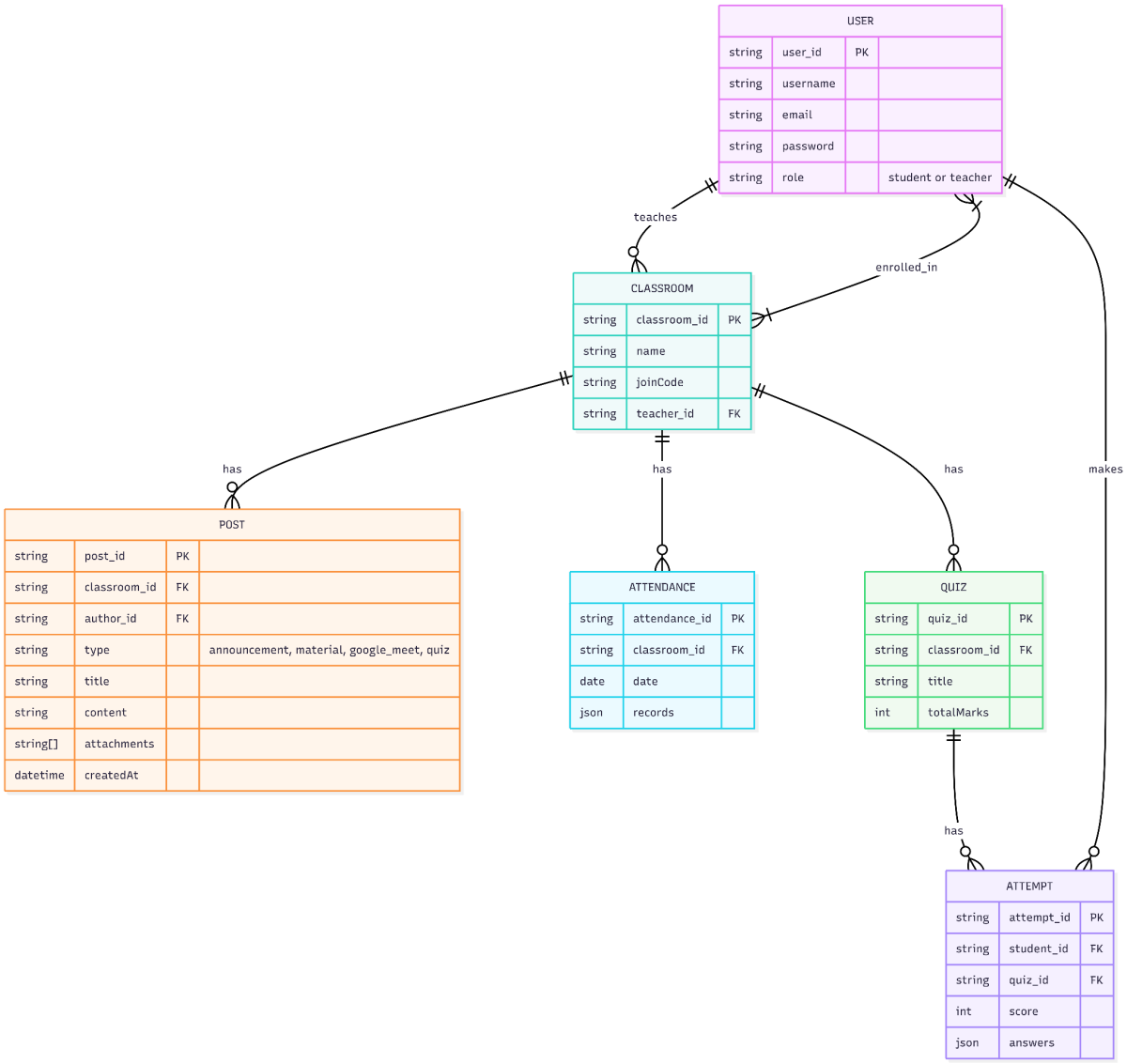

Figure 3.1: Entity Relationship Diagram of Collify

4. User Management

4.1 Registration and Login

The User Management module forms the foundation of the Collify platform by providing secure and role-based access to the system. Users can register as either a **Student** or a **Teacher**, and the system differentiates functionality based on their role.

Key Features:

- New users can register by providing a unique username, email address, password, and selecting their role (Student or Teacher).
- Passwords are securely hashed using bcrypt before storing in the database.
- Users can log in using their email and password.
- Upon successful login, a JWT (JSON Web Token) is generated and sent to the client for authentication in subsequent requests.
- Role-based access control is implemented using JWT payload, ensuring teachers have access to creation and management features, while students have read and participation rights.

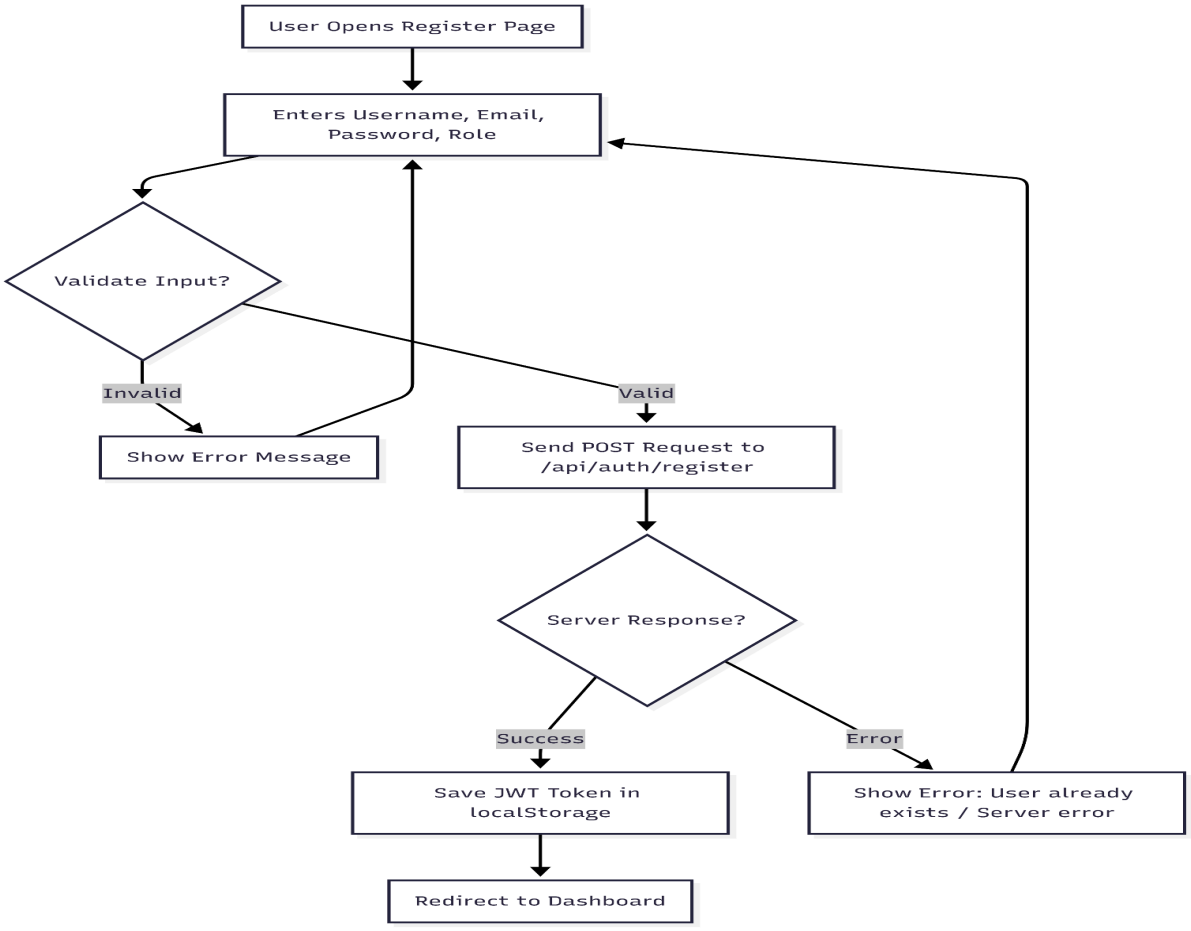

Figure 4.1: Registration Route Flow Diagram

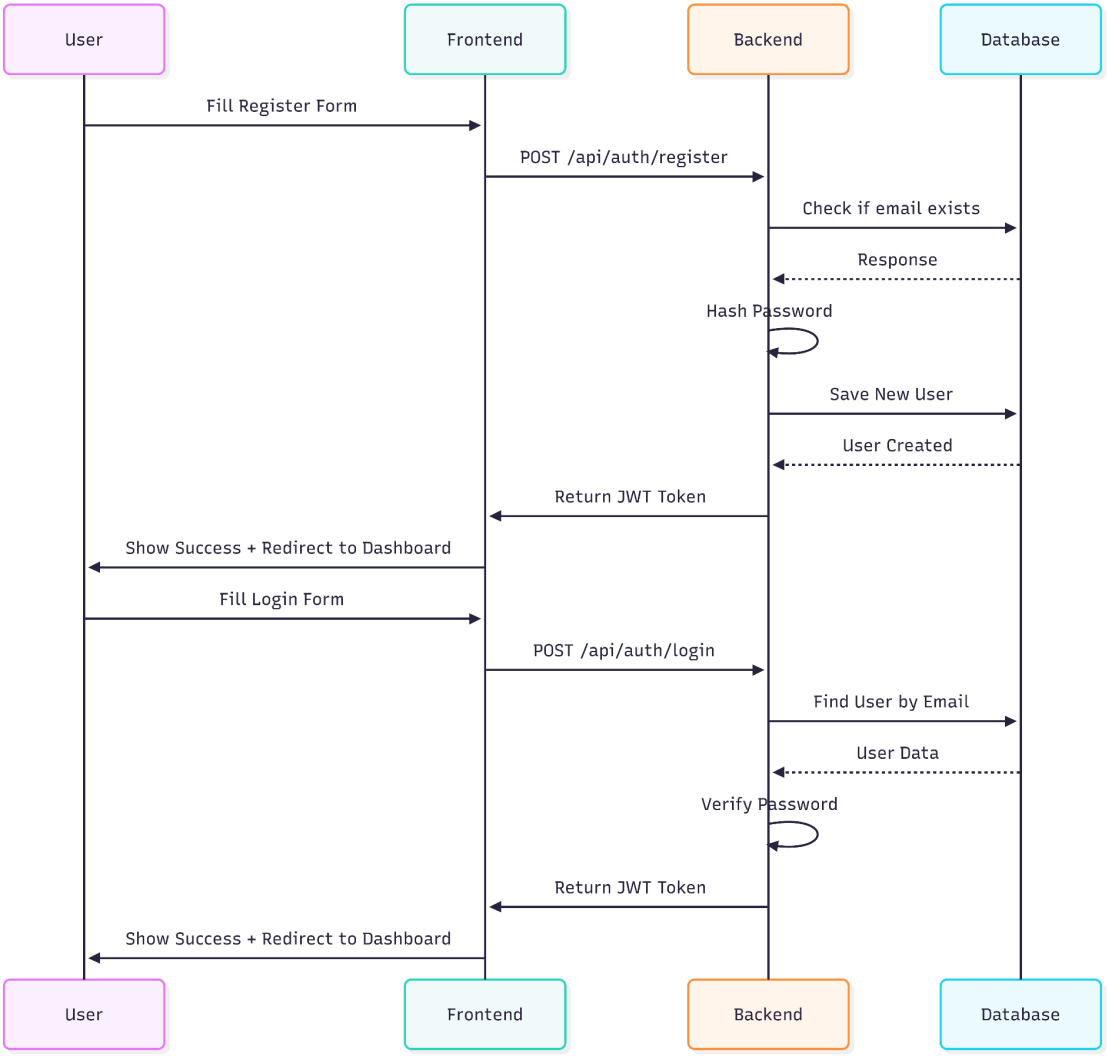

Figure 4.2 : Sequence Diagram

4.2 User Profiles

Each user has a profile that stores basic information such as username, email, and role. The system currently uses a simple user model, with scope for future enhancements such as profile picture upload, bio, and activity history.

The user profile is linked to classrooms (as teacher or student), posts (as author), and quiz attempts. This central user entity enables seamless role-based functionality across the entire platform.

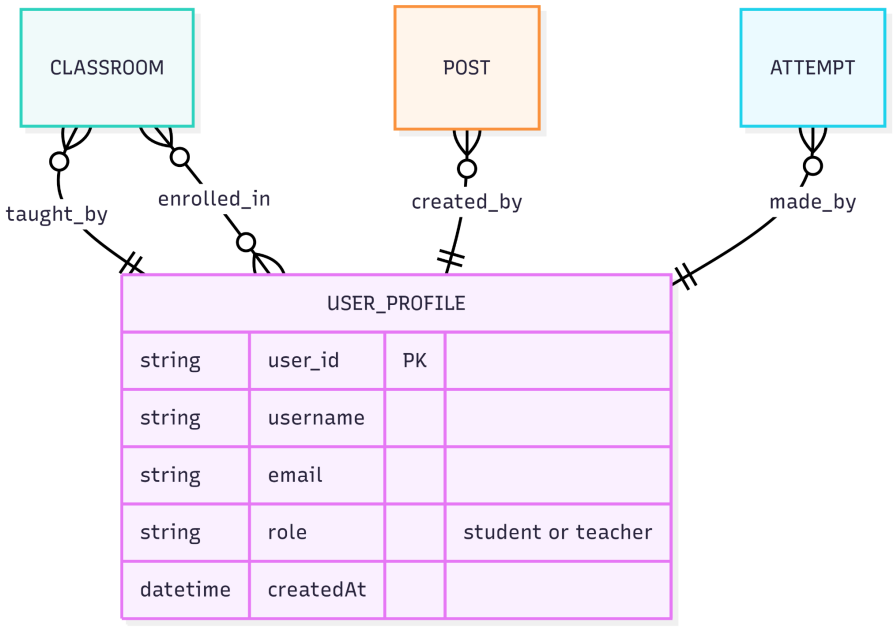

Figure 4.3 : User Profile

4.3 Status Updates

The Status Updates module in Collify allows users to view real-time progress and activity related to their classrooms and assessments. This feature provides transparency and keeps both teachers and students informed about the current state of various activities within the platform.

Key Features Implemented:

- **Real-time Classroom Status**: Users can see whether they are a teacher or student in a particular classroom.
- **Post Activity Status**: Displays the latest posts and announcements made by the teacher with timestamps.
- **Quiz Status**: Shows the status of quizzes (e.g., "Upcoming", "Active", "Completed") along with scores for attempted quizzes.
- **Attendance Status**: Students can view their attendance summary (Present/Absent) for recent sessions, while teachers can see overall class attendance trends.
- **Recent Activity Feed**: A summarized view of recent actions such as new posts, quiz attempts, and classroom joins.

Current Implementation:

- Status updates are currently displayed on the Dashboard and Classroom Detail pages.
- The system fetches and shows dynamic data such as join codes, post creation timestamps, and basic quiz/attendance summaries.
- Role-based visibility is enforced — teachers see management-related status, while students see participation-related status.

---

5. Backend Design

The backend of Collify is built using **Node.js** with the **Express.js** framework, providing a robust, scalable, and RESTful API layer. It handles all business logic, data processing, authentication, and communication with the MongoDB database.

5.1 API Endpoints

The backend exposes well-organized RESTful API endpoints grouped by functionality:

Authentication Routes

- POST /api/auth/register – User registration with role selection
- POST /api/auth/login – User login and JWT token generation

Classroom Routes

- POST /api/classrooms/create – Create a new classroom (Teacher only)
- POST /api/classrooms/join – Join a classroom using join code
- GET /api/classrooms/my-classrooms – Fetch all classrooms for the logged-in user
- GET /api/classrooms/:classroomId – Get details of a specific classroom

Post Routes

- POST /api/classrooms/:classroomId/posts – Create a new post (Teacher only)
- GET /api/classrooms/:classroomId/posts – Get all posts for a classroom

Future Routes (Planned)

- Quiz creation and management
- Attendance marking and retrieval
- Coding problem submission and evaluation

All routes are protected using JWT-based middleware. Role-based authorization (using teacherOnly middleware) ensures that sensitive operations like creating classrooms and posts are restricted to teachers only.

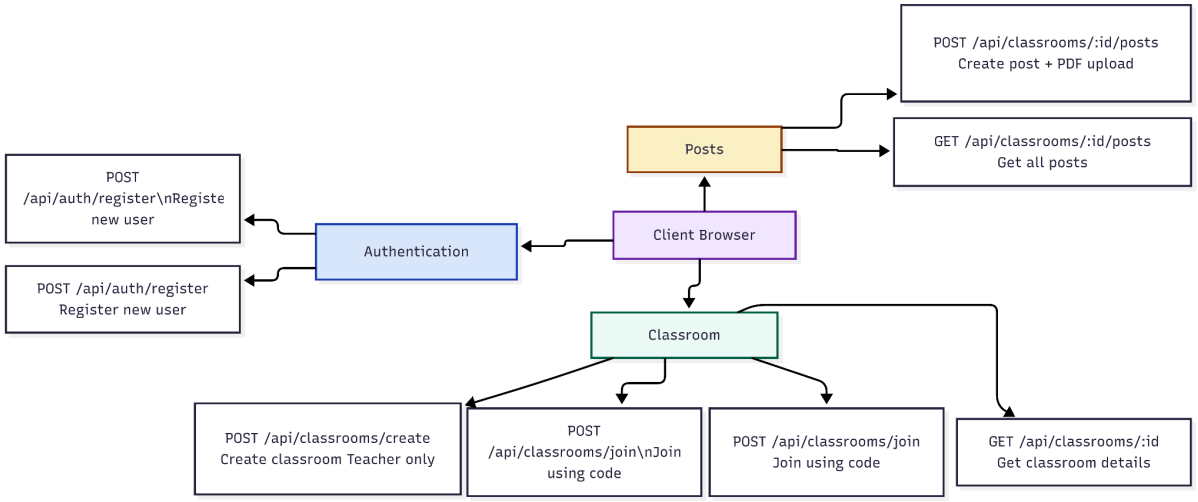

Figure 5.1 : API Endpoints

5.2 Database Schema

The system uses **MongoDB** with **Mongoose** as the ODM (Object Document Mapper). The schema is designed to be flexible yet structured:

- **User Schema**: Stores user credentials and role
- **Classroom Schema**: Contains classroom details and references to teacher and enrolled students
- **Post Schema**: Supports different post types with optional fields (meetLink, attachments, quiz_id)
- **Attendance Schema**: Stores daily attendance records per classroom
- **Quiz & Attempt Schemas**: For assessment functionality

Relationships are maintained using ObjectId references and populated queries for efficient data retrieval.

5.3 Authentication and Authorization

Authentication is implemented using **JWT (JSON Web Tokens)**. Upon successful login or registration, a token containing the user's id and role is issued. This token is sent in the Authorization header (Bearer <token>) for all protected routes.

- **protect middleware**: Verifies the JWT token and attaches user data to req.user
- **teacherOnly middleware**: Ensures only users with role: 'teacher' can access teacher-specific routes

Passwords are hashed using **bcrypt** before storage. The system follows secure practices such as token expiration and proper error handling for unauthorized access.

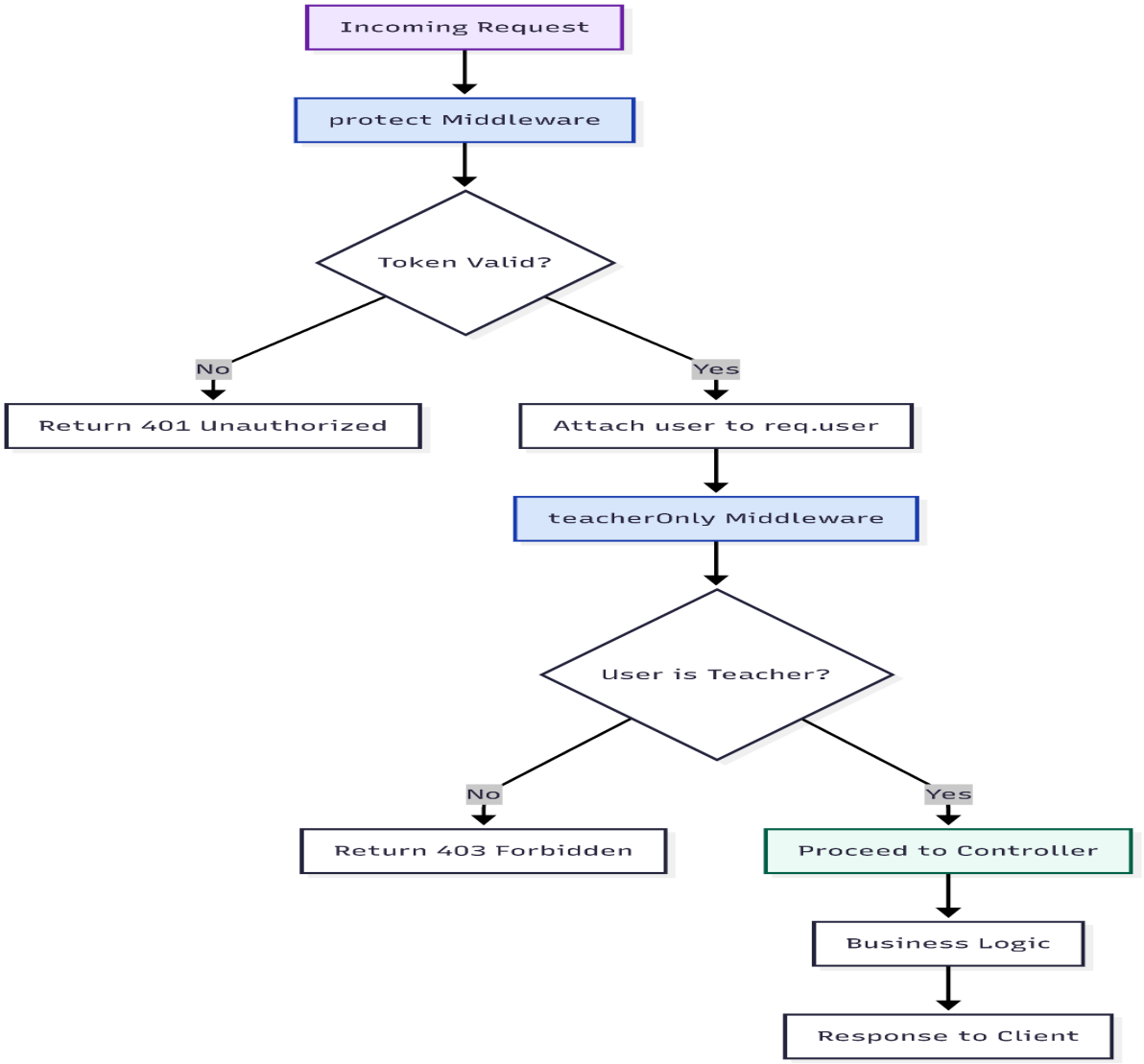

Figure 5.2 : Middleware flow in Collify Backend

6. Frontend Design

The frontend of Collify is developed using **Next.js 14** with the App Router, providing a fast, responsive, and modern user interface. The design focuses on simplicity, clarity, and role-based visibility to deliver an intuitive experience for both teachers and students.

6.1 User Interface

The user interface follows a clean and minimalistic design with a consistent layout across all pages. Key pages include:

- **Landing Page**: Welcomes users and provides clear options to Login or Register.
- **Dashboard**: Displays the list of classrooms in a responsive grid layout. Teachers can create new classrooms, while students can join using a join code.
- **Classroom Detail Page**: Shows classroom information, posts, and role-specific actions. Teachers can create posts, while students can view content and attempt quizzes.
- **Login & Register Pages**: Simple and secure forms with proper validation and error handling.

The interface uses a light color scheme with indigo accents for primary actions, ensuring good readability and a professional look.

6.2 Component Structure

The frontend is built using reusable React components for better maintainability:

- **Navbar**: Global navigation component showing logo, role, and login/logout options.
- **ClassroomCard**: Reusable card component used in the dashboard to display classroom details.
- **PostCard**: Displays individual posts with title, content, author, date, and attachments.
- **CreatePostForm**: Form component for teachers to create new posts with title, content, type, and PDF attachments.

All components are organized in the components/ folder and follow a modular structure for easy reuse and future expansion.

6.3 Responsive Design

The application is designed to be fully responsive:

- On mobile devices, the classroom grid collapses to a single column.
- Forms and buttons adjust gracefully across different screen sizes.
- Navigation remains accessible on all devices.

Pure CSS with CSS Modules is used for styling, giving full control over the design without external dependencies.

---

7. Testing

Testing is an essential part of the Collify development process to ensure reliability, security, and a smooth user experience. The project follows a structured testing approach covering different levels of the application.

7.1 Unit Testing

Unit tests are written to verify the correctness of individual functions and components in isolation.

- **Backend Unit Tests**: Test individual functions such as password hashing, JWT token generation, and input validation.
- **Frontend Unit Tests**: Test React components like ClassroomCard, form validation, and state management logic.
- Tools used: Jest (planned for future phases).

7.2 Integration Testing

Integration tests ensure that different modules work correctly together.

- Testing the interaction between API routes and the database (e.g., creating a classroom and verifying it is saved correctly).
- Verifying that protected routes correctly reject requests without a valid JWT token.
- Testing file upload flow (Multer + Cloudinary) and ensuring attachments are properly linked to posts.

7.3 End-to-End Testing

End-to-End (E2E) testing simulates real user scenarios from start to finish.

- **User Registration and Login Flow**: Register a new user, login, and verify successful redirection to the dashboard.
- **Classroom Flow**: Teacher creates a classroom → Student joins using the join code → Both users see the classroom in their dashboard.
- **Post Creation Flow**: Teacher creates a post with PDF attachment → Student can view the post and download the attachment.
- **Role-based Access**: Verify that students cannot access teacher-only features (e.g., create post button is hidden).

Current Testing Status:

- Manual testing has been performed on all major flows (registration, login, classroom creation, joining, post creation).
- Automated testing framework (Jest + React Testing Library) is planned for the next phase to improve test coverage and regression testing.

---

8. Conclusion

Collify successfully demonstrates the development of a modern, scalable, and user-friendly Classroom Management System that integrates essential academic functionalities into a single unified platform. By combining classroom management, content sharing, automated assessments, and attendance tracking, the system effectively reduces the dependency on multiple disconnected tools and streamlines the overall teaching-learning process.

The platform's role-based architecture ensures a clear separation of responsibilities, allowing teachers to efficiently manage classrooms, create content, and evaluate student performance, while students benefit from a structured and distraction-free learning environment. The use of technologies such as Next.js, Node.js, Express, and MongoDB ensures high performance, scalability, and maintainability, making the system suitable for real-world deployment in educational institutions.

Additionally, the implementation of secure authentication mechanisms using JWT and bcrypt highlights the system's focus on data security and user privacy. The modular design of both frontend and backend components allows for easy future enhancements and feature expansion.

In conclusion, Collify provides a strong foundation for a comprehensive Learning Management System (LMS). With planned future enhancements such as a coding judge, advanced analytics, and improved attendance tracking, the platform has the potential to evolve into a complete digital ecosystem that enhances engagement, improves efficiency, and supports modern education in a meaningful way.

---

9. Future Enhancements

While Collify already delivers a robust set of features for classroom management and assessment, there are several opportunities to further enhance its capabilities and make it a more comprehensive and intelligent learning platform.

One of the key planned enhancements is the integration of a **Coding Judge System**. This feature will allow teachers to assign programming problems, especially for Data Structures and Algorithms (DSA), and automatically evaluate student submissions against predefined test cases. This will significantly benefit technical courses and improve hands-on learning.

Another major improvement involves **Advanced Analytics and Reporting**. Teachers will be able to access detailed insights into student performance, such as quiz-wise analysis, class averages, participation trends, and attendance patterns. Visual dashboards with charts and graphs can help educators make data-driven decisions to improve learning outcomes.

The platform can also be enhanced by introducing **Real-Time Notifications**. Students and teachers can receive instant alerts for new posts, quiz deadlines, and classroom updates through push notifications or email integration, ensuring that no important information is missed.

A **Live Class Integration** feature can be developed to embed video conferencing tools directly within the platform. Instead of sharing external links, teachers can conduct live sessions seamlessly inside Collify, improving accessibility and user experience.

Additionally, implementing a **Discussion Forum or Chat System** within classrooms would encourage better interaction between students and teachers. This will foster collaborative learning and allow students to ask questions, share ideas, and engage in academic discussions.

Another important enhancement is **Role Expansion**, such as introducing an *Admin role* to manage platform-wide settings, monitor activity, and maintain system integrity across multiple institutions.

To further improve usability, **Mobile Application Development** (using React Native or Flutter) can be considered. A dedicated mobile app would provide a smoother experience for students attempting quizzes and checking updates on the go.

Finally, enhancing **Security and Performance** through features like rate limiting, API throttling, data encryption, and caching mechanisms (e.g., Redis) will make the system more robust and production-ready.

With these enhancements, Collify has the potential to evolve into a fully-featured, enterprise-level Learning Management System that not only simplifies classroom operations but also enriches the overall educational experience

---

10. References

The development of Collify is based on established technologies, frameworks, and best practices in full-stack web development. The following references were used for implementation and conceptual understanding:

1. Next.js Documentation – Official documentation for building optimized React applications with server-side rendering and App Router.
2. Node.js Documentation – Core concepts for building scalable backend services using JavaScript runtime.
3. Express.js Documentation – Guidelines for creating RESTful APIs and handling server-side routing.
4. MongoDB Documentation – Reference for NoSQL database design, queries, and data modeling.
5. Mongoose Documentation – Object Data Modeling (ODM) library used for schema design and database interaction.
6. JSON Web Token (JWT) Documentation – Concepts and implementation of secure authentication and authorization.
7. bcrypt Documentation – Best practices for password hashing and security.
8. Cloudinary Documentation – Integration for cloud-based file storage and media handling.
9. AWS Documentation – Cloud infrastructure services for deployment and hosting.
10. REST API Design Guidelines – Standard practices for designing scalable and maintainable APIs.

These resources provided the foundational knowledge required to design, implement, and optimize the Collify platform.

---

11. Appendix

The appendix section provides supplementary information that supports the understanding and implementation of the Collify system.

A. Sample API Request/Response

Login Request:

POST /api/auth/login

```
{
 "email": "user@example.com",
 "password": "password123"
}
```

Login Response:

```
{
 "token": "jwt_token_here",
 "user": {
   "id": "12345",
   "username": "Arun",
   "role": "student"
 }
}
```

B. Sample Classroom Object (MongoDB)

```
{
 "_id": "class123",
 "name": "Data Structures",
 "joinCode": "ABC123",
 "teacher": "user123",
 "students": ["user456", "user789"],
 "createdAt": "2026-03-20T10:00:00Z"
```

}

C. Folder Structure (Simplified)

Backend:

/backend

├── /controllers

├── /models

├── /routes

├── /middleware

├── server.js

Frontend:

/frontend

├── /app

├── /components

├── /styles

├── package.json
